# Random Movement

**Random Movement**

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

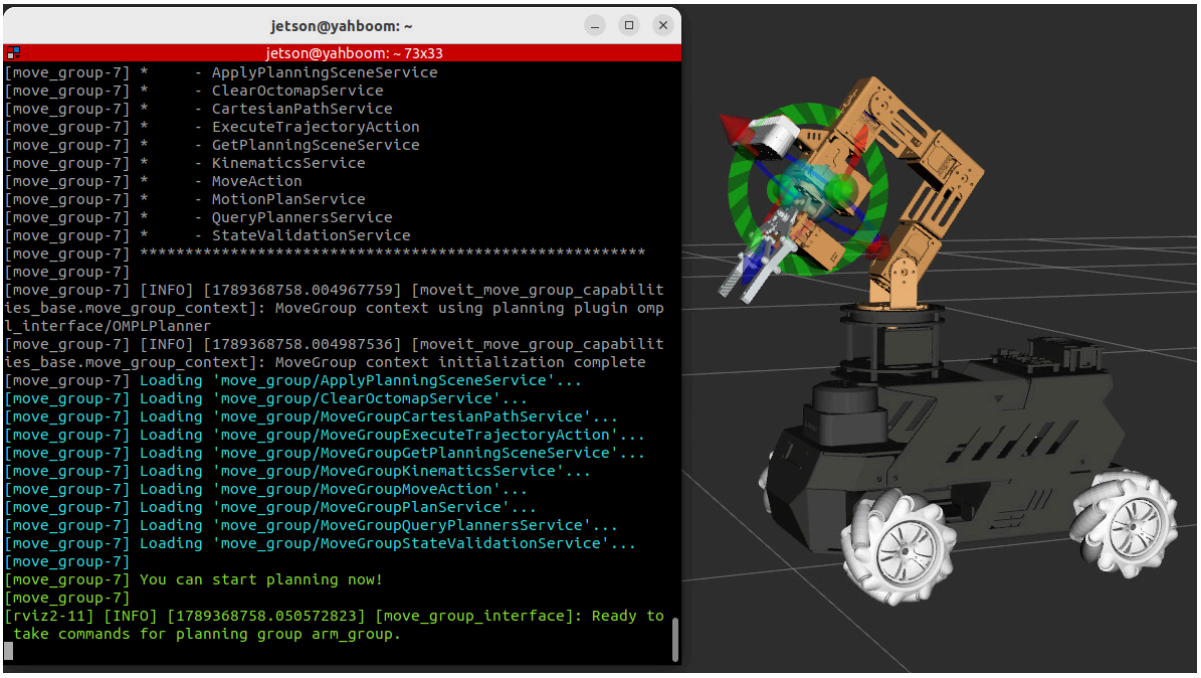
## 1. Course Content

This section explains how to use the MoveIt2 library to make the robot arm in RViz randomly move to a certain pose.

## 2. Preparation

```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
```

Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:



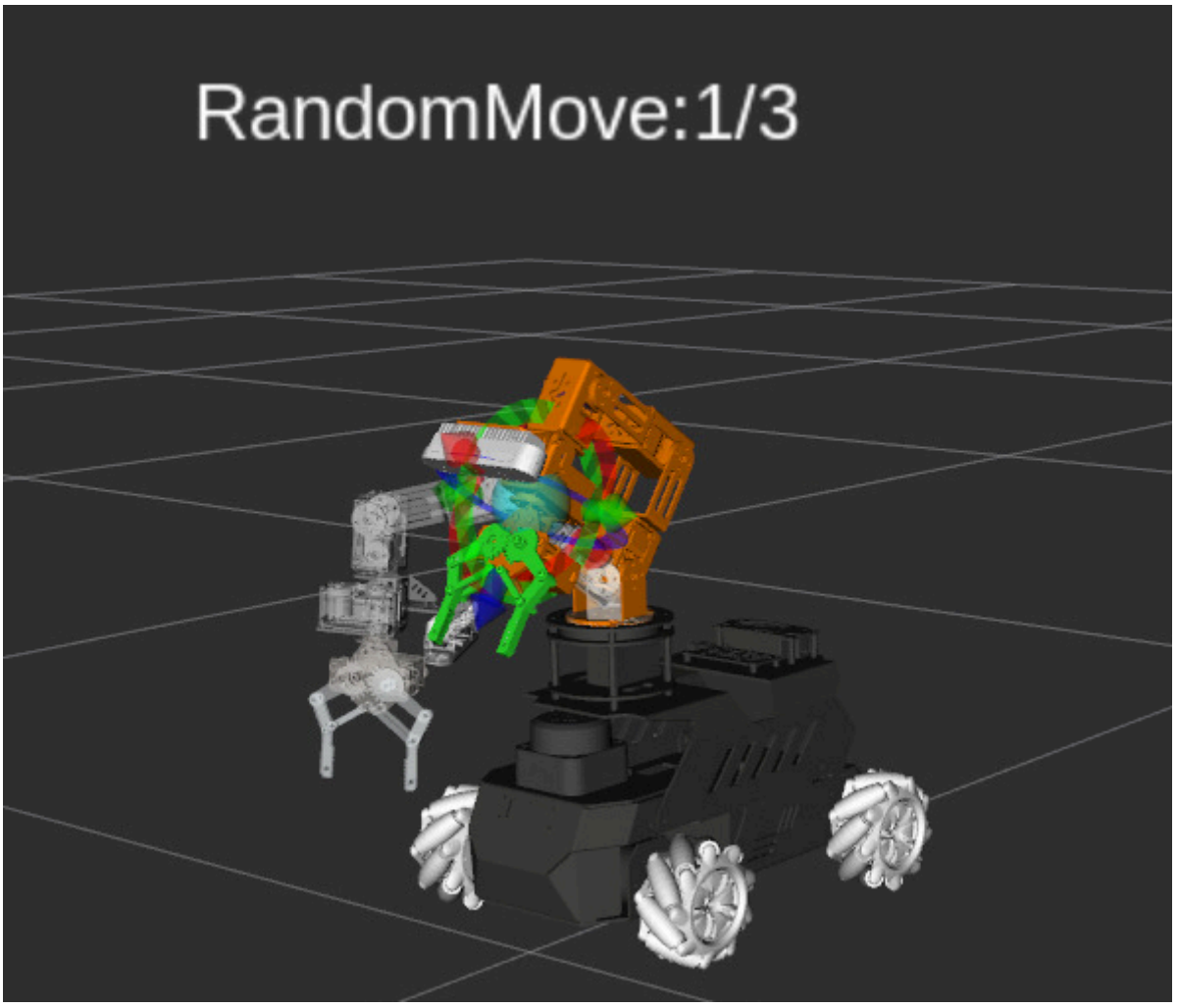

## 3. Case Demonstration

Then enter the following command in the terminal to launch the random movement program:

```
ros2 run moveit_demo random_move
```

After the program runs, the robot arm in RViz will perform 3 random position movements, as shown in the figure below:


RandomMove:0/3


RandomMove:1/3


RandomMove:3/3

# 4. Core Source Code Analysis

Program code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/moveit_demo/src/random_move.cpp
```

**initialize function description:**

- `MoveGroupInterface(shared_from_this(), "arm_group")`: Creates a MoveGroup control interface, bound to the `arm_group` planning group, used to control the robot arm motion planning.
- `PlanningSceneInterface`: Creates a planning scene interface, used to manage the collision objects in the scene.
- `setNumPlanningAttempts(10)`: Sets the number of planning attempts per call to 10, improving the planning success rate.
- `setPlanningTime(10.0)`: Sets the maximum time for a single planning attempt to 10 seconds.
- `setMaxVelocityScalingFactor(1.0)`: Sets the maximum velocity scaling factor to 1.0, i.e., using the full speed of the planned motion.
- `setMaxAccelerationScalingFactor(1.0)`: Sets the maximum acceleration scaling factor to 1.0, i.e., using the full acceleration of the planned motion.
- `MoveItVisualTools`: Creates an RViz visualization tool for displaying text and markers in RViz.
- `deleteAllMarkers()`: Clears all existing markers in RViz.
- `loadRemoteControl()`: Loads the RViz remote control buttons to facilitate debugging interaction.
- `setNamedTarget("up")`: Sets the target to the preset "up" pose.
- `plan(init_plan)`: Plans a trajectory to the "up" pose and returns whether it succeeded.
- `execute(init_plan)`: Executes the planned trajectory, moving the robot arm to the "up" pose.
- `create_wall_timer(std::chrono::seconds(10), ...)`: Creates a timer with a 10-second interval, triggering a random movement every 10 seconds.

**move_to_random_target function description:**

- `move_count_ >= 3`: After 3 random movements are completed, cancels the timer and shuts down the node to end the program.
- `setRandomTarget()`: Randomly generates a set of joint angles within the joint space as the target pose.
- `plan(my_plan)`: Plans a trajectory to the random target, checking whether a collision-free path exists.
- `execute(my_plan)`: If planning succeeds, executes the trajectory and drives the robot arm to the random pose.
- `for (int attempt = 0; attempt < 20; attempt++)`: Tries the random target up to 20 times until a successfully plannable path is found.

- `timer_->cancel()` and `rclcpp::shutdown()`: After all 3 movements are complete, stops the timer and shuts down the node.

**publish_status_text function description:**

- `deleteAllMarkers()`: Clears old markers before publishing new text each time.
- `Eigen::Isometry3d::Identity()`: Creates an identity transformation matrix as the text display pose.
- `text_pose.translation().z() = 0.5`: Displays the text at a height of z=0.5 m.
- `publishText(text_pose, text, rvt::WHITE, rvt::XLARGE)`: Publishes the status text in RViz in white extra-large font.
- `trigger()`: Triggers a render update so that the text is visible in RViz.

```
class RandomMove : public rclcpp::Node
{
public:
  RandomMove()
    : Node("random_target_move"), move_count_(0)
  {
    RCLCPP_INFO(this->get_logger(), "Initializing RandomMoveIt2Control.");
  }

  void initialize()
  {
    // Initialize MoveGroupInterface after shared_ptr is fully constructed
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>(
      shared_from_this(), "arm_group");
    planning_scene_interface_ =
std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    move_group_interface_->setNumPlanningAttempts(10);
    move_group_interface_->setPlanningTime(10.0);
    move_group_interface_->setMaxVelocityScalingFactor(1.0);
    move_group_interface_->setMaxAccelerationScalingFactor(1.0);

    // Initialize RViz visual tools for text display
    visual_tools_ = std::make_shared<moveit_visual_tools::MoveItVisualTools>(
      shared_from_this(),
      move_group_interface_->getPlanningFrame(),
      "random_move_markers");
    visual_tools_->deleteAllMarkers();
    visual_tools_->loadRemoteControl();
    publish_status_text("RandomMove:0/3");

    // Move arm to "up" position for initialization
    moveit::planning_interface::MoveGroupInterface::Plan init_plan;
    move_group_interface_->setNamedTarget("up");
    bool init_success = (move_group_interface_->plan(init_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
    if (init_success)
    {
```

```
      RCLCPP_INFO(this->get_logger(), "Init arm succeeded.");
      move_group_interface_->execute(init_plan);
    }
    else
    {
      RCLCPP_ERROR(this->get_logger(), "Init arm failed!");
    }

    timer_ = this->create_wall_timer(
      std::chrono::seconds(10),
      std::bind(&RandomMove::move_to_random_target, this));
  }

private:
  void move_to_random_target()
  {
    if (move_count_ >= 3)
    {
      RCLCPP_INFO(this->get_logger(), "Random movement ended.");
      publish_status_text("RandomMove:Ended");
      timer_->cancel();
      rclcpp::shutdown();
      return;
    }

    move_count_++;
    RCLCPP_INFO(this->get_logger(), "Attempting random movement %d/3.",
move_count_);
    publish_status_text("RandomMove:"+std::to_string(move_count_)+"/3");

    bool found_valid_target = false;
    moveit::planning_interface::MoveGroupInterface::Plan my_plan;

    // Retry up to 20 times to find a collision-free random target
    for (int attempt = 0; attempt < 20; attempt++)
    {
      move_group_interface_->setRandomTarget();

      bool plan_success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);

      if (plan_success)
      {
        found_valid_target = true;
        RCLCPP_INFO(this->get_logger(), "Random movement %d: Planning succeeded,
moving the arm.", move_count_);
        move_group_interface_->execute(my_plan);
        break;
      }
      else
      {
        RCLCPP_WARN(this->get_logger(),
          "Random movement %d: Planning attempt %d failed, retrying...",
move_count_, attempt + 1);
      }
    }
```

```
    if (!found_valid_target)
    {
      RCLCPP_ERROR(this->get_logger(),
        "Random movement %d: Failed to find a collision-free target after 20
attempts.", move_count_);
    }

    // Check if all 3 movements are done after this attempt
    if (move_count_ >= 3)
    {
      publish_status_text("RandomMove:Ended");
      timer_->cancel();
      rclcpp::shutdown();
    }
  }

  void publish_status_text(const std::string& text)
  {
    if (visual_tools_)
    {
      visual_tools_->deleteAllMarkers();
      Eigen::Isometry3d text_pose = Eigen::Isometry3d::Identity();
      text_pose.translation().z() = 0.5;
      visual_tools_->publishText(text_pose, text, rvt::WHITE, rvt::XLARGE);
      visual_tools_->trigger();
    }
  }

  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
  std::shared_ptr<moveit_visual_tools::MoveItVisualTools> visual_tools_;
  rclcpp::TimerBase::SharedPtr timer_;
  int move_count_;
};
```